

数据库系统第三—四章—SQL 详细学习笔记

2026 年 6 月 15 日

源课件：数据库系统 PPT-ch3-4-sqlnew.pdf（共 237 页）
风格：考点导向 · 中英双语术语 · 页码引用 · 完整 SQL 示例

目录

1 SQL 概述与历史 [p.2–5]	2
1.1 SQL 的起源与发展 [p.3–4]	2
1.2 SQL 语言分类（DDL/DML/DCL） [p.5]	2
1.3 DDL 的功能 [p.6]	2
2 SQL 数据类型 [p.7–8]	3
2.1 字符类型 [p.7]	3
2.2 数值类型 [p.7–8]	3
2.3 日期时间类型 [p.8]	3
2.4 其他类型 [p.8]	3
3 DDL —数据定义语言 [p.9–14]	4
3.1 CREATE DATABASE [p.9]	4
3.2 CREATE TABLE [p.10–12]	4
3.2.1 基本语法 [p.10–11]	4
3.2.2 完整性约束 [p.11]	4
3.2.3 SC 表实例 [p.12]	5
3.3 ALTER TABLE / DROP TABLE [p.13]	5
3.4 银行示例数据库的完整模式 [p.14]	5
4 SQL 查询基础结构：SELECT-FROM-WHERE [p.15–36]	7
4.1 基本查询结构 [p.15]	7
4.2 SELECT 子句 [p.18–20]	7
4.2.1 基本使用 [p.18]	7
4.2.2 DISTINCT vs ALL [p.19]	7
4.2.3 星号与算术表达式 [p.20]	7
4.3 WHERE 子句 [p.21–22]	8
4.3.1 BETWEEN [p.22]	8

4.4	FROM 子句 [p.23]	8
4.5	LIKE 一字符串模式匹配 [p.34–35]	8
4.6	AS 一重命名 [p.31–33]	9
4.6.1	属性重命名 [p.31]	9
4.6.2	元组变量 (Tuple Variables) [p.33]	9
4.7	ORDER BY 一排序 [p.36]	9
4.8	SELECT INTO 一创建新表 [p.80–81]	9
4.9	WHERE 子句常用查询条件总结 [p.82]	9
5	连接查询 (Join) [p.26–30, 92–104]	11
5.1	连接操作分类 [p.26]	11
5.2	传统连接 (WHERE 方式) [p.27]	11
5.3	自然连接 (Natural Join) [p.28–30]	11
5.4	内连接 (Inner Join) [p.92–100]	12
5.5	外连接 (Outer Join) [p.92–102]	12
5.6	自连接 (Self Join) [p.103–104]	13
6	集合运算 (Set Operations) [p.39–40]	14
6.1	UNION / INTERSECT / EXCEPT [p.39]	14
6.2	保留重复元组的版本 [p.39]	14
7	聚合函数与分组 (Aggregate Functions & GROUP BY) [p.41–47]	15
7.1	聚合函数 [p.41–42]	15
7.2	GROUP BY 子句 [p.43–44]	15
7.3	HAVING 子句 [p.43–45]	15
7.4	SELECT 语句完整结构 [p.47]	16
7.5	COMPUTE 子句 (SQL Server 特有) [p.90–91]	16
8	空值处理 (NULL Values) [p.48–50]	17
8.1	NULL 的概念 [p.48]	17
8.2	三值逻辑 (Three-Valued Logic) [p.49]	17
8.3	聚合函数与 NULL [p.50]	17
9	嵌套子查询 (Nested Subqueries) [p.51–69]	18
9.1	子查询概念 [p.51]	18
9.2	IN / NOT IN [p.52–54]	18
9.3	SOME / ALL [p.57–60]	18
9.3.1	SOME [p.57–58]	18
9.3.2	ALL [p.59–60]	19
9.4	EXISTS / NOT EXISTS [p.61–69]	19
9.5	UNIQUE [p.70–71]	19
9.6	FROM 子句中的子查询 [p.72]	20

10 数据修改 (Modification of the Database) [p.121–128]	21
10.1 DELETE —删除元组 [p.121–122]	21
10.2 INSERT —插入元组 [p.123–124]	21
10.3 UPDATE —更新元组 [p.125–126]	22
11 视图 (Views) [p.109–120, 127–129]	23
11.1 视图的概念 [p.109, 115]	23
11.2 CREATE VIEW [p.111–112]	23
11.3 视图的依赖关系 [p.113–114]	23
11.4 物化视图 (Materialized View) [p.116]	23
11.5 视图的优缺点 [p.117]	24
11.6 创建视图的限制 [p.118–119]	24
11.7 通过视图更新数据 [p.127–129]	24
12 用户自定义类型与域 [p.130–131]	25
12.1 CREATE TYPE / CREATE DOMAIN [p.130]	25
12.2 域的约束 [p.131]	25
12.3 大对象类型 [p.132]	25
13 完整性约束 (Integrity Constraints) [p.133–144]	26
13.1 概述 [p.133]	26
13.2 NOT NULL 约束 [p.134–135]	26
13.3 PRIMARY KEY 约束 [p.134]	26
13.4 FOREIGN KEY 约束 [p.134, 139–141]	26
13.5 UNIQUE 约束 [p.136]	26
13.6 CHECK 约束 [p.137–138]	26
13.7 断言 (Assertion) [p.142–144]	27
14 授权 (Authorization) [p.145–152]	28
14.1 授权类型 [p.145]	28
14.2 GRANT —授予权限 [p.146–147, 149–150]	28
14.3 REVOKE —收回权限 [p.148, 151–152]	28
15 SQL 流程控制 [p.153–175]	30
15.1 变量 [p.153–160]	30
15.1.1 局部变量 [p.154–157]	30
15.1.2 全局变量 [p.158–160]	30
15.2 IF...ELSE 语句 [p.161–162]	30
15.3 BEGIN...END 语句块 [p.163]	30
15.4 CASE 语句 [p.167–168]	30
15.5 WHILE 循环 [p.170–171]	31
15.6 GOTO / RETURN [p.172–175]	31

16 存储过程 (Stored Procedures) [p.176–213]	32
16.1 概念与特点 [p.177–181]	32
16.2 创建存储过程 [p.185–189]	32
16.3 变量声明与赋值 [p.189–191]	33
16.4 流程控制 (MySQL) [p.195–209]	33
16.5 DELIMITER [p.192–194]	33
16.6 注释与权限 [p.210]	33
16.7 条件处理 (HANDLER) [p.211–213]	33
17 触发器 (Triggers) [p.214–229]	34
17.1 触发器概念 [p.214]	34
17.2 触发器语法分析 [p.215–223]	34
17.2.1 创建限制 [p.215]	34
17.2.2 触发器名称 [p.216]	34
17.2.3 触发时间 [p.217]	34
17.2.4 触发事件 [p.218]	34
17.2.5 FOR EACH ROW [p.220]	34
17.2.6 OLD 和 NEW 引用 [p.221–223]	34
17.3 触发器示例 [p.224–228]	35
17.4 触发器的缺点 [p.229]	35
18 游标 (Cursor) [p.230–235]	36
18.1 游标概念 [p.230–231]	36
18.2 游标使用步骤 [p.232–234]	36
18.3 游标示例 [p.235]	36
19 综合示例与补充 [p.73–91, 55–56]	38
19.1 学生数据库完整示例 [p.73–76]	38
19.2 综合 SQL 练习示例 [p.55–56]	38
19.3 GROUP BY + HAVING 综合 [p.88–89]	39
20 易错点与考点总结	40
20.1 DDL 相关易错点	40
20.2 DML 查询易错点	40
20.3 子查询易错点	40
20.4 连接易错点	40
20.5 数据修改易错点	40
20.6 其他考点	41
21 SQL 关键词速查表	42

1 SQL 概述与历史 [p.2-5]

1.1 SQL 的起源与发展 [p.3-4]

- 1974 年由 Boyce 和 Chamber 提出，最初称为 **Sequel** (Structured English QUery Language) [p.3]
- 1975-1979 年由 IBM San Jose 研究室在 System R 上首次实现 [p.3]
- 后更名为 **SQL** (Structured Query Language) [p.3]
- ANSI/ISO 标准演进：
 - SQL-86, SQL-89
 - SQL-92 (SQL2)
 - SQL:1999 (SQL3): 面向对象支持, 抽象数据类型, 递归, 触发器
 - SQL:2003, SQL:2008
 - SQL:2011 (ISO/IEC 9075:2011)
 - SQL:2016 (ISO/IEC 9075:2016): 行模式识别、JSON 对象支持、多态表函数 [p.4]

1.2 SQL 语言分类 (DDL/DML/DCL) [p.5]

- SQL 是集 DDL、DML 和 DCL 于一体的数据库语言，由 9 个关键词引导 [p.5]
- **DDL (Data Definition Language)** 语句引导词: CREATE, ALTER, DROP [p.5]
 - 模式定义和删除: Database, Table, View, Index, 完整性约束等
 - 也包括定义对象 (RowType 行对象, Type 列对象)
- **DML (Data Manipulation Language)** 语句引导词: INSERT, DELETE, UPDATE, SELECT [p.5]
 - 各种更新与检索操作: 直接输入/SubQuery 输入
 - 各种复杂条件检索: 连接查找、模糊查找、分组查找、嵌套查找
 - 各种聚集操作: 求平均、求和等; 分组聚集、分组过滤
- **DCL (Data Control Language)** 语句引导词: GRANT, REVOKE [p.5]

易错点: DDL 与 DML 的区分是常考点。DDL 操作的是数据库对象的结构 (表、索引、视图等), DML 操作的是数据内容 (增删改查)。

1.3 DDL 的功能 [p.6]

DDL 不仅可以指定一组关系，还可以包含以下信息 [p.6]:

- 每个关系的模式 (Schema)
- 每个属性的值域 (Domain of values)
- 完整性约束 (Integrity constraints)
- 每个关系要维护的索引集 (Indices)
- 安全性和授权信息
- 每个关系在磁盘上的物理存储结构

2 SQL 数据类型 [p.7-8]

2.1 字符类型 [p.7]

- `char(n)`: 固定长度字符串, 用户指定长度 `n` [p.7]
- `varchar(n)`: 可变长度字符串, 最大长度 `n` [p.7]
- `nchar(n)`, `nvarchar(n)`: Unicode (双字节编码), 1-4000 字符 [p.7]
- `text`: 可变长度字符串, 1-2GB, 等同于 `varchar(max)` [p.7]

2.2 数值类型 [p.7-8]

- `int`: 整数 (4 字节) [p.7]
- `smallint`: 小整数 (2 字节) [p.7]
- `bigint`: 大整数 (8 字节) [p.7]
- `tinyint`: (1 字节), `bit`: (1 位) [p.7]
- `numeric(p,d)` / `decimal(p,d)`: 定点数, `p` 为总有效数字位数 (不含小数点), `d` 为小数位数 [p.7]
 - 例: `numeric(5,3)`, 如 3.14159 存储为 3.142; 5123.23 会溢出 (左边只能 2 位数) [p.7]
- `real`, `double precision`: 浮点/双精度浮点 (双精度 8 字节) [p.8]
- `float(n)`: 浮点数 (单精度 4 字节), 用户指定精度至少 `n` 位 [p.8]

2.3 日期时间类型 [p.8]

- `datetime` (8B): `yyyy.mm.dd h:m:s`, 精确到分钟 [p.8]
- `smalldatetime` (4B) [p.8]
- `date`: 日期 (如 2003-09-12) [p.8]
- `time`: 时间 (如 23:15:003) [p.8]

2.4 其他类型 [p.8]

- `money` (8B): 货币类型 [p.8]
- `xml` (<2GB): 存储 XML 数据 [p.8]
- `binary(n)`: 1-8000B [p.8]
- `image` (<2GB): 存储图像 [p.8]
- `timestamp` (8B) [p.8]
- `boolean`: SQL:1999 引入, 值 `true` / `false` / `unknown` [p.8]
- `sql_variant` ≤ 8016bit [p.8]
- `cursor`: 游标, 查询结果数据集 [p.8]
- `table`: 表格式数据 [p.8]

3 DDL —数据定义语言 [p.9–14]

3.1 CREATE DATABASE [p.9]

创建数据库文件的基本语法 [p.9]:

```
CREATE DATABASE database_name
ON PRIMARY
( NAME = student_data,
  FILENAME = 'C:\...\student.mdf',
  SIZE = 10MB,
  MAXSIZE = 50MB,
  FILEGROWTH = 10% )
LOG ON
( NAME = student_log,
  FILENAME = 'C:\...\student.ldf',
  SIZE = 1MB,
  MAXSIZE = 20MB,
  FILEGROWTH = 10% )
```

3.2 CREATE TABLE [p.10–12]

3.2.1 基本语法 [p.10–11]

```
CREATE TABLE r (
  A1 D1, A2 D2, ..., An Dn,
  integrity-constraint1,
  ...,
  integrity-constraintk
);
```

其中: r 为关系名, A_i 为属性名, D_i 为属性域 (数据类型) [p.10]

示例: 创建 `instructor` 表 [p.10]:

```
CREATE TABLE instructor (
  ID          CHAR(5),
  name        VARCHAR(20) NOT NULL,
  dept_name   VARCHAR(20),
  salary      NUMERIC(8,2)
);
```

3.2.2 完整性约束 [p.11]

创建表时可在属性后或表级指定约束:

- NOT NULL: 非空约束 [p.11]
- PRIMARY KEY ($A_1, \dots, A_n$): 主键约束。SQL-92 起自动确保 NOT NULL, SQL-89 需显式声明 [p.11]

- FOREIGN KEY (A1, ..., An) REFERENCES r: 外键约束 [p.11]
带主键和外键的完整示例 [p.11]:

```
CREATE TABLE instructor (
  ID          CHAR(5),
  name        VARCHAR(20) NOT NULL,
  dept_name   VARCHAR(20),
  salary      NUMERIC(8,2),
  PRIMARY KEY (ID),
  FOREIGN KEY (dept_name) REFERENCES department
);
```

3.2.3 SC 表实例 [p.12]

学生-课程表 SC, 约束关系 [p.12]:

- 学生关系: student(sid, sname, Ssex, age), 主码 id
- 课程关系: course(course_no, course_name, credits)
- SC 关系: sc(student_id, course_no, grade)

```
CREATE TABLE sc (
  student_id CHAR(8),
  course_no   CHAR(4),
  grade       INTEGER,
  PRIMARY KEY (student_id, course_no),
  FOREIGN KEY (student_id) REFERENCES student(id),
  FOREIGN KEY (course_no) REFERENCES course(course_no)
);
```

注意: 外键的列名可以不同, 但域必须相同 [p.12]。

3.3 ALTER TABLE / DROP TABLE [p.13]

- ALTER TABLE r ADD A D: 增加属性 A (域 D)。所有已有元组新属性值为 null [p.13]
— 例: ALTER TABLE instructor ADD tel CHAR(11);
- ALTER TABLE r DROP COLUMN A: 删除属性 A [p.13]
- ALTER TABLE sc ALTER COLUMN sno CHAR(4);: 修改属性域 [p.13]
- DROP TABLE r: 删除表 (删除所有相关信息) [p.13]
- DROP DATABASE student: 删除数据库 [p.13]
- 注意: 很多数据库不支持删除属性 (dropping of attributes) [p.13]

3.4 银行示例数据库的完整模式 [p.14]

```
CREATE TABLE student (
  ID          VARCHAR(5),
  name        VARCHAR(20) NOT NULL,
  dept_name   VARCHAR(20),
```



```
tot_cred    NUMERIC(3,0),
PRIMARY KEY (ID),
FOREIGN KEY (dept_name) REFERENCES department
);

CREATE TABLE course (
  course_id  VARCHAR(8),
  title      VARCHAR(50),
  dept_name  VARCHAR(20),
  credits    NUMERIC(2,0),
  PRIMARY KEY (course_id),
  FOREIGN KEY (dept_name) REFERENCES department
);

CREATE TABLE takes (
  ID          VARCHAR(5),
  course_id   VARCHAR(8),
  sec_id      VARCHAR(8),
  semester    VARCHAR(6),
  year        NUMERIC(4,0),
  grade       VARCHAR(2),
  PRIMARY KEY (ID, course_id, sec_id, semester, year),
  FOREIGN KEY (ID) REFERENCES student,
  FOREIGN KEY (course_id, sec_id, semester, year) REFERENCES section
);
```

注意: sec_id 可以从主键中删除,以确保学生不能在同一学期注册同一课程的两个 section [p.14]

4 SQL 查询基础结构: *SELECT-FROM-WHERE* [p.15–36]

4.1 基本查询结构 [p.15]

SQL 查询的基本形式 [p.15]:

```
SELECT A1, A2, ..., An
FROM   r1, r2, ..., rm
WHERE  P;
```

- A_i 代表属性 (attribute)
- R_i 代表关系 (relation)
- P 是谓词 (predicate)

该查询等价于关系代数表达式: $\Pi_{A_1, A_2, \dots, A_n}(\sigma_P(r_1 \times r_2 \times \dots \times r_m))$ [p.15]

SQL 查询的结果是一个关系 (relation) [p.15]

4.2 *SELECT* 子句 [p.18–20]

4.2.1 基本使用 [p.18]

- *SELECT* 列出查询结果中要出现的属性, 对应关系代数的投影 (Π) [p.18]
- SQL 命名不分大小写: $\text{Name} \equiv \text{NAME} \equiv \text{name}$ [p.18]
- 例: 找出所有老师的名字 [p.18]

```
SELECT name
FROM instructor;
```

4.2.2 *DISTINCT* vs *ALL* [p.19]

- SQL 允许重复行 [p.19]
- *DISTINCT*: 强制消除重复元组 [p.19]

```
SELECT DISTINCT name FROM instructor;
```

- *ALL*: 保留重复行 (默认行为) [p.19]

```
SELECT ALL name FROM instructor;
```

易错点: *SELECT* 默认不消除重复 (即隐含 *ALL*), 只有显式使用 *DISTINCT* 才会去重。

4.2.3 星号与算术表达式 [p.20]

- *SELECT **: 表示全部属性 [p.20]
- *SELECT* 子句可以包含算术表达式 ($+$, $-$, $*$, $/$), 操作常数或元组属性 [p.20]

```
SELECT ID, name, salary/12
FROM instructor;
-- 结果中 salary 列的值除以 12
```

4.3 WHERE 子句 [p.21–22]

- WHERE 指定结果必须满足的条件，对应关系代数的选择 (σ) [p.21]
- 例：找出 Comp. Sci. 系中薪水 >80000 的老师 [p.21]

```
SELECT name
FROM instructor
WHERE dept_name = 'Comp. Sci.' AND salary > 80000;
```

- 比较结果可以用逻辑连接词 AND, OR, NOT 组合 [p.21]
- 比较可以应用于算术表达式的结果 [p.21]

4.3.1 BETWEEN [p.22]

查找薪水在 15000~20000 之间的员工 [p.22]:

```
SELECT name, salary
FROM instructor
WHERE salary BETWEEN 15000 AND 20000;
-- 等价于: WHERE salary >= 15000 AND salary <= 20000
```

4.4 FROM 子句 [p.23]

- FROM 列出查询中涉及的关系，对应关系代数的笛卡尔积 [p.23]
- SELECT * FROM instructor, teaches; 生成所有可能的 instructor-teaches 对 [p.23]
- 笛卡尔积本身用处不大，但与 WHERE 子句结合使用才有意义 [p.23]
- 例：找出 2010 年有课的老师姓名与所在系 [p.23]

```
SELECT name, dept_name
FROM instructor, teaches
WHERE instructor.id = teaches.id AND year = 2010;
```

4.5 LIKE 一字符串模式匹配 [p.34–35]

- LIKE 使用两个特殊字符进行模式匹配 [p.34]:
 - %: 匹配任意子串（包括空串）
 - _: 匹配任意单个字符
- 查找街道中包含“岳麓”的客户 [p.34]:

```
SELECT customer_name
FROM customer
WHERE customer_street LIKE '%岳麓%';
```

- ESCAPE: 转义字符 [p.35]
 - 匹配名字中含“%”符号: LIKE 'Main%' ESCAPE '\'
- NOT LIKE: 表示不匹配 [p.35]
- 其他字符串函数: 拼接 (||)、大小写转换、子串提取、求长度等 [p.35]

4.6 AS 一重命名 [p.31–33]

4.6.1 属性重命名 [p.31]

```
SELECT customer_name, borrower.loan_number AS loan_id, amount
FROM borrower, loan
WHERE borrower.loan_number = loan.loan_number;
```

4.6.2 元组变量 (Tuple Variables) [p.33]

元组变量在 FROM 子句中通过 AS 定义, AS 关键字可选 [p.33]:

```
SELECT customer_name, T.loan_number, S.amount
FROM borrower AS T, loan AS S
WHERE T.loan_number = S.loan_number;
-- AS 可以省略: borrower T, loan S 等同
```

自连接示例: 找出所有比位于”长沙”的分行拥有更多资产的分行名称 [p.33]

```
SELECT DISTINCT T.branch_name
FROM branch AS T, branch AS S
WHERE T.assets > S.assets AND S.branch_city = '长沙';
```

4.7 ORDER BY 一排序 [p.36]

- ORDER BY A1, A2, ... ASC/DESC [p.36]
- ASC 为升序 (默认), DESC 为降序 [p.36]
- 例: 按字母顺序列出中行铁道分行贷款的所有客户 [p.36]

```
SELECT DISTINCT customer_name
FROM borrower, loan
WHERE borrower.loan_number = loan.loan_number
      AND branch_name = '中行铁道分行'
ORDER BY customer_name ASC;
```

4.8 SELECT INTO 一创建新表 [p.80–81]

可从查询结果创建新表 [p.80]:

```
SELECT column_name1, column_name2, ...
INTO new_table
FROM table_name;
```

4.9 WHERE 子句常用查询条件总结 [p.82]

- 比较: <, <=, >, >=, =, <>
- 确定范围: BETWEEN A AND B 与 NOT BETWEEN A AND B

- 确定集合: IN, NOT IN
- 字符匹配: LIKE, NOT LIKE
- 空值: IS NULL, IS NOT NULL
- 存在性: EXISTS, NOT EXISTS
- 多重条件: AND, OR, NOT

5 连接查询 (Join) [p.26–30, 92–104]

5.1 连接操作分类 [p.26]

在 SQL 中, 连接 (Join) 操作可以分为以下几种 [p.26]:

- 自然连接 (Natural Join)
- 内连接 (Inner Join)
- 外连接 (Outer Join)

5.2 传统连接 (WHERE 方式) [p.27]

```
-- 找所有教过课的老师姓名和所教课程编号
SELECT name, course_id
FROM instructor, teaches
WHERE instructor.ID = teaches.ID;

-- 找出2020年有课的老师姓名与所在系
SELECT name, dept_name
FROM instructor, teaches
WHERE instructor.id = teaches.id AND year = 2020;
```

5.3 自然连接 (Natural Join) [p.28–30]

自然连接在所有公共属性上匹配具有相同值的元组, 并保留每个公共列的一份副本 [p.28]:

```
SELECT * FROM instructor NATURAL JOIN teaches;
```

对比两种写法 [p.29]:

```
-- 方式1: WHERE 方式
SELECT name, course_id
FROM instructor, teaches
WHERE instructor.ID = teaches.ID;

-- 方式2: NATURAL JOIN
SELECT name, course_id
FROM instructor NATURAL JOIN teaches;
```

自然连接的危险 [p.30]: 注意名称相同但不相关的属性, 它们会被不正确地等同起来。

错误版本 (让 course.dept_name = instructor.dept_name):

```
-- 错误!
SELECT name, title
FROM instructor NATURAL JOIN teaches NATURAL JOIN course;
```

正确版本:

```
-- 方式1: 混合使用
SELECT name, title
```

```
FROM instructor NATURAL JOIN teaches, course
WHERE teaches.course_id = course.course_id;

-- 方式2: 使用 JOIN ... USING
SELECT name, title
FROM (instructor NATURAL JOIN teaches)
JOIN course USING(course_id);
```

5.4 内连接 (Inner Join) [p.92-100]

内连接用比较运算符比较两个表中列值, 将满足连接条件的行组合起来 [p.93]:

基本语法 [p.98]:

```
-- 外连接效果等同于 WHERE
loan INNER JOIN borrower ON
    loan.loan_number = borrower.loan_number;

-- 自然内连接
loan NATURAL INNER JOIN borrower;

-- 示例: 查询课程名称及任课老师信息
SELECT teacher_info.teacher_id, teacher_info.name,
       lesson_info.course_name
FROM lesson_info INNER JOIN teacher_info
ON (lesson_info.course_id = teacher_info.course_id);
```

注意: 自然连接对结果集的冗余列进行限制, 删除连接表中的重复列 [p.93]

5.5 外连接 (Outer Join) [p.92-102]

外连接通过在结果中创建包含空值元组的方式, 保留在连接中丢失的元组 [p.93]。

- 左外连接 (LEFT OUTER JOIN): 对左边表不加限制 [p.101]

```
loan LEFT OUTER JOIN borrower ON
    loan.loan_number = borrower.loan_number;
```

- 右外连接 (RIGHT OUTER JOIN): 对右边表不加限制 [p.96, 101]

```
loan NATURAL RIGHT OUTER JOIN borrower;
```

- 全外连接 (FULL OUTER JOIN): 对两个表都不加限制 [p.97, 101]

```
loan FULL OUTER JOIN borrower USING (loan_number);
```

全外连接应用示例——找出仅有账户或仅有贷款的客户 [p.97]:

```
SELECT customer_name
FROM (depositor NATURAL FULL OUTER JOIN borrower)
WHERE account_number IS NULL OR loan_number IS NULL;
```

外连接总结表:

连接类型	左表	右表	结果
INNER JOIN	限制	限制	仅匹配行
LEFT OUTER JOIN	不限制	限制	左表全保留
RIGHT OUTER JOIN	限制	不限制	右表全保留
FULL OUTER JOIN	不限制	不限制	两表全保留

5.6 自连接 (Self Join) [p.103-104]

数据表通过自连接实现自身与自身连接, 可以看作一张表的两个副本之间的连接 [p.103]:

```
SELECT aliat1.column_name1, aliat1.column_name2, ...  
FROM table_name aliat1, table_name aliat2  
WHERE [aliat1.column_name join_operator aliat2.column_name];
```

关键点: 在自连接中, 同一个表应给出不同的别名 [p.103]

6 集合运算 (Set Operations) [p.39-40]

6.1 UNION / INTERSECT / EXCEPT [p.39]

与关系代数的 $\cup$, $\cap$, $-$ 对应, 自动去掉重复 [p.39]:

- **UNION**: 并集, 自动去重 [p.40]

```
(SELECT customer_name FROM depositor)
UNION
(SELECT customer_name FROM borrower);
```

- **INTERSECT**: 交集, 自动去重 [p.40]

```
(SELECT customer_name FROM depositor)
INTERSECT
(SELECT customer_name FROM borrower);
```

- **EXCEPT**: 差集, 自动去重 [p.40]

```
(SELECT customer_name FROM depositor)
EXCEPT
(SELECT customer_name FROM borrower);
```

6.2 保留重复元组的版本 [p.39]

如果不想去重, 使用 ALL 版本 [p.39]:

- **UNION ALL**: 元组在 r 中出现 m 次, 在 s 中出现 n 次, 则在结果中出现 m+n 次
- **INTERSECT ALL**: 出现 $\min(m, n)$ 次
- **EXCEPT ALL**: 出现 $\max(0, m-n)$ 次

易错点: UNION 默认去重, UNION ALL 保留重复。这常常是性能相关的考虑点。

7 聚合函数与分组 (Aggregate Functions & GROUP BY) [p.41-47]

7.1 聚合函数 [p.41-42]

聚合函数操作在一列值的多重集上并返回一个值 [p.41]:

- AVG: 平均值
- MIN: 最小值
- MAX: 最大值
- SUM: 总和
- COUNT: 值的数量

示例 [p.42]:

```
-- 检索中行铁道支行平均账户余额
SELECT AVG(balance)
FROM account
WHERE branch_name = '中行铁道支行';

-- 统计银行存款人关系中的元组数量
SELECT COUNT(*)
FROM depositor;

-- 统计银行存款人数量
SELECT COUNT(customer_name)
FROM depositor;
```

7.2 GROUP BY 子句 [p.43-44]

GROUP BY 按一定条件对查询结果分组, 再对每组计算统计信息 [p.43]:

```
SELECT branch_name, COUNT(DISTINCT customer_name)
FROM depositor, account
WHERE depositor.account_number = account.account_number
GROUP BY branch_name;
```

关键规则 [p.44]: 除聚集函数外, SELECT 中出现的属性必须在 GROUP BY 中出现。

7.3 HAVING 子句 [p.43-45]

- WHERE 是对元组限定条件, HAVING 是对分组限定条件 [p.45]
- WHERE 条件作用于分组前, HAVING 条件作用于分组后 [p.45]
- 例: 找出平均账户存款余额 >2000 的银行 [p.45]

```
SELECT branch_name, AVG(balance)
FROM account
GROUP BY branch_name
HAVING AVG(balance) > 2000;
```

易错点 (重要考点) —— WHERE vs HAVING 区别:

- WHERE 在 GROUP BY 之前过滤行（不能使用聚合函数）
- HAVING 在 GROUP BY 之后过滤组（可以使用聚合函数）
- WHERE 消除的行不参与分组；HAVING 消除的分组不输出

7.4 SELECT 语句完整结构 [p.47]

```
SELECT attribute list          -- 输出列
FROM table list                -- 输入表
WHERE selection-condition      -- 过滤行
GROUP BY grouping attribute(s) -- 分组
HAVING grouping condition     -- 过滤组
ORDER BY {attribute ASC|DESC} pairs; -- 排序
```

注意事项 [p.47]:

- 只有 SELECT 和 FROM 是必须的，其余可选
- 子句的顺序不能改变
- FROM 指定的表数有限制

7.5 COMPUTE 子句 (SQL Server 特有) [p.90–91]

COMPUTE 生成合计作为附加的汇总列，出现在结果集的最后。与 BY 一起使用时进行分类汇总 [p.90]:

```
SELECT tech_title, salary
FROM teacher_info
WHERE tech_title = N'讲师'
ORDER BY tech_title
COMPUTE SUM(salary);
```

8 空值处理 (NULL Values) [p.48–50]

8.1 NULL 的概念 [p.48]

- NULL 表示未知值或值不存在 [p.48]
- 使用 IS NULL 来检查空值 (不能用 = NULL) [p.48]

```
SELECT loan_number
FROM loan
WHERE amount IS NULL;    -- 注意: 不能用 WHERE amount = NULL
```

- 任何涉及 NULL 的算术表达式结果都为 NULL [p.48]
- 例: 5 + null 返回 null [p.48]

8.2 三值逻辑 (Three-Valued Logic) [p.49]

SQL:1999 引入 boolean 类型, 值为: TRUE, FALSE, UNKNOWN [p.49]

- 任何与 NULL 的比较都返回 unknown [p.49]
- 例: 5 < null, null <> null, null = null 均返回 unknown
- 三值逻辑的真值表 [p.49]:
 - **OR**: (unknown OR true) = true, (unknown OR false) = unknown, (unknown OR unknown) = unknown
 - **AND**: (true AND unknown) = unknown, (false AND unknown) = false, (unknown AND unknown) = unknown
 - **NOT**: (NOT unknown) = unknown
- 如果 WHERE 子句谓词结果为 unknown, 则视为 false (该行不被选中) [p.49]

8.3 聚合函数与 NULL [p.50]

- 除 COUNT(*) 外, 所有聚合操作都忽略 NULL 值 [p.50]
- COUNT(*) 包括所有元组 (含 NULL 行的元组) [p.50]
- COUNT(属性名) 不包括属性为 NULL 的元组 [p.50]
- 例: SELECT SUM(amount) FROM loan; 中 null 的 amount 不参与求和 [p.50]

易错点:

- 判断 NULL 必须用 IS NULL 而不能用 = NULL
- COUNT(*) 与 COUNT(列名) 的区别: 后者忽略 NULL
- NULL 在 WHERE 条件中的行为: 涉及 NULL 的比较结果为 unknown, 视为 false

9 嵌套子查询 (Nested Subqueries) [p.51–69]

9.1 子查询概念 [p.51]

- 子查询 (Subquery) 是嵌套在另一个查询中的 SELECT-FROM-WHERE 表达式 [p.51]
- 常见用途 [p.51]:
 - 测试集合成员关系 (IN)
 - 集合比较 (>some, >all 等)
 - 集合基数 (EXISTS, 是否存在元组)
 - 元组唯一性 (UNIQUE)

9.2 IN / NOT IN [p.52–54]

测试元组是否在 (或不在) 集合中 [p.52]:

```
-- 既有账户又有贷款的客户
SELECT DISTINCT customer_name
FROM borrower
WHERE customer_name IN (SELECT customer_name FROM depositor);

-- 有贷款但没有账户的客户
SELECT DISTINCT customer_name
FROM borrower
WHERE customer_name NOT IN (SELECT customer_name FROM depositor);
```

多属性 IN [p.53]: 可以同时测试多个属性是否在子查询结果中。

使用 IN 查询指定集合 [p.54]:

```
SELECT id, course_id, grade
FROM takes
WHERE course_id IN ('0001', '0002');
-- 等价于: WHERE course_id = '0001' OR course_id = '0002'
```

9.3 SOME / ALL [p.57–60]

9.3.1 SOME [p.57–58]

$F <\text{comp}> \text{SOME } r \Leftrightarrow \exists t \in r \text{ 使得 } (F <\text{comp}> t)$ [p.57]

= SOME 等价于 IN, 但 $<>$ SOME 不等于 NOT IN [p.57]

```
-- 找至少比位于长沙的某些银行资产大的银行
SELECT branch_name
FROM branch
WHERE assets > SOME
  (SELECT assets FROM branch WHERE branch_city = '长沙');
```

9.3.2 ALL [p.59–60]

$F <\text{comp}> \text{ALL } r \Leftrightarrow \forall t \in r \text{ 使得 } (F <\text{comp}> t)$ [p.59]

$<> \text{ALL}$ 等价于 NOT IN , 而 $= \text{ALL}$ 等价于 IN [p.59]

```
-- 找出资产大于位于长沙的所有分行的分行
SELECT branch_name
FROM branch
WHERE assets > ALL
  (SELECT assets FROM branch WHERE branch_city = '长沙');
```

9.4 EXISTS / NOT EXISTS [p.61–69]

- $\text{EXISTS } r \Leftrightarrow r \neq \emptyset$ (测试集合是否不为空) [p.61]
- $\text{NOT EXISTS } r \Leftrightarrow r = \emptyset$ (测试集合是否为空) [p.61]

关键应用——”全部”的双重否定模式： ”找到在长沙所有分行都有账户的客户” [p.62]:

```
SELECT DISTINCT S.customer_name
FROM depositor AS S
WHERE NOT EXISTS (
  (SELECT branch_name FROM branch WHERE branch_city = '长沙')
  EXCEPT
  (SELECT R.branch_name
   FROM depositor AS T, account AS R
   WHERE T.account_number = R.account_number
        AND S.customer_name = T.customer_name)
);
```

核心思想: $X - Y = \emptyset \Leftrightarrow X \subseteq Y$, 用 NOT EXISTS 实现”所有”语义 [p.62]

相关子查询 (Correlated Subquery) vs 不相关子查询 (Uncorrelated Subquery):

- 不相关子查询: 子查询独立于外层查询, 只执行一次
- 相关子查询: 子查询引用外层查询的属性, 对外层每一行重新执行
- EXISTS 通常用于相关子查询, 对每行测试子查询是否非空
- 效率考虑: 相关子查询效率较低, 因为对外层每行都要执行

9.5 UNIQUE [p.70–71]

测试子查询结果中是否没有重复元组 [p.70]:

```
-- 最多拥有一个账户的客户
SELECT T.customer_name
FROM depositor AS T
WHERE UNIQUE (
  SELECT R.customer_name
  FROM account, depositor AS R
  WHERE T.customer_name = R.customer_name
        AND R.account_number = account.account_number
```

```
    AND account.branch_name = '中行铁道支行'
);

-- 至少拥有两个账户的客户 (使用 NOT UNIQUE)
SELECT DISTINCT T.customer_name
FROM depositor AS T
WHERE NOT UNIQUE (
    SELECT R.customer_name
    FROM account, depositor AS R
    WHERE ...
);
```

9.6 FROM 子句中的子查询 [p.72]

SQL 允许在 FROM 子句中使用子查询表达式 [p.72]:

```
SELECT branch_name, avg_balance
FROM (SELECT branch_name, AVG(balance)
      FROM account
      GROUP BY branch_name)
      AS branch_avg (branch_name, avg_balance)
WHERE avg_balance > 1200;
```

注意: 这种方式可以替代 HAVING 子句, 因为临时表 branch_avg 的属性可以直接在 WHERE 中使用 [p.72]

10 数据修改 (Modification of the Database) [p.121–128]

10.1 DELETE —删除元组 [p.121–122]

语法:

```
DELETE FROM r
WHERE P;
```

示例 [p.121]:

```
-- 删除中行铁道支行的所有账户元组
DELETE FROM account
WHERE branch_name = '中行铁道支行';

-- 删除位于长沙市的所有分支机构的账户
DELETE FROM account
WHERE branch_name IN
  (SELECT branch_name FROM branch
   WHERE branch_city = '长沙市');
```

注意 DELETE 执行顺序 [p.122]: 删除余额低于平均余额的账户时, SQL 的处理方式:

1. 首先计算 avg balance 并找出所有要删除的元组
2. 然后删除上面找到的所有元组 (不重新计算 avg 或重新测试)

10.2 INSERT —插入元组 [p.123–124]

语法:

```
INSERT INTO r VALUES (...);
```

示例 [p.123]:

```
-- 插入一个元组
INSERT INTO account
VALUES ('A-9732', '铁道支行', 1200);

-- 或者等价指定列顺序
INSERT INTO account (branch_name, balance, account_number)
VALUES ('铁道支行', 1200, 'A-9732');

-- 插入 NULL 值
INSERT INTO account
VALUES ('A-777', '铁道支行', NULL);
```

用子查询插入 [p.124]: 为铁道支行的所有贷款客户提供 200 元的储蓄账户作为礼物:

```
INSERT INTO account
SELECT loan_number, branch_name, 200
FROM loan
WHERE branch_name = '铁道支行';
```



```
INSERT INTO depositor
SELECT customer_name, loan_number
FROM loan, borrower
WHERE branch_name = '铁道支行'
AND loan.account_number = borrower.account_number;
```

重要：SELECT-FROM-WHERE 语句在任何结果插入关系之前完全求值，避免了诸如 INSERT INTO table1 SELECT * FROM table1 产生的问题 [p.124]

10.3 UPDATE —更新元组 [p.125–126]

语法：

```
UPDATE r
SET col1 = expr1, ...
WHERE P;
```

使用 CASE 语句 [p.126]：余额 >10000 的账户增加 6%，其余增加 5%：

```
UPDATE account
SET balance = CASE
    WHEN balance <= 10000 THEN balance * 1.05
    ELSE balance * 1.06
END;
```

注意：用两条 UPDATE 语句时顺序很重要（先更新 >10000 的 6%，再更新 ≤10000 的 5%） [p.125]

11 视图 (Views) [p.109–120, 127–129]

11.1 视图的概念 [p.109, 115]

- 视图提供了一种向某些用户隐藏某些数据的机制（安全性） [p.109]
- 任何不属于概念模型但对用户可见的”虚拟关系”称为视图 [p.109]
- 视图是基于某个查询结果的一个虚拟表，只是用来查看数据的窗口 [p.115]
- 数据库中只存放视图的定义，不存放视图对应的数据，数据仍存放在基表中 [p.115]
- 当基表数据变化时，视图查询出来的数据也随之改变 [p.115]

11.2 CREATE VIEW [p.111–112]

语法 [p.111]:

```
CREATE VIEW v AS <query expression>;
```

创建视图时，只将查询表达式存储在数据库中，并不存储查询结果关系。使用视图时才每次执行查询计算结果 [p.111]。

示例——创建银行分支机构及其客户的视图 [p.112]:

```
CREATE VIEW all_customer AS
(
  SELECT branch_name, customer_name
  FROM depositor, account
  WHERE depositor.account_number = account.account_number
)
UNION
(
  SELECT branch_name, customer_name
  FROM borrower, loan
  WHERE borrower.loan_number = loan.loan_number);

-- 使用视图查询
SELECT customer_name
FROM all_customer
WHERE branch_name = '中行铁道支行';
```

11.3 视图的依赖关系 [p.113–114]

- 一个视图可以在定义另一个视图的表达式中使用 [p.113]
- 视图 v1 直接依赖于 v2，如果 v2 用于定义 v1 的表达式 [p.113]
- 如果 v1 依赖于 v2 或存在依赖路径，则 v1 依赖于 v2 [p.113]
- 如果视图依赖自身，则称其为递归视图 [p.113]

视图展开 (View Expansion): 将视图定义中的视图关系用其定义表达式替换，直到表达式中不再存在视图关系。只要视图定义不是递归的，这个循环就会终止 [p.114]。

11.4 物化视图 (Materialized View) [p.116]

- 物化视图: 视图结果关系存储在数据库中 [p.116]
- 当定义视图的基本关系改变时，视图也随之修改 [p.116]

- 物化视图维护：保持物化视图总是处于最新状态 [p.116]
- 优点：对频繁使用视图的情况可以提高效率
- 缺点：增加了存储与更新维护开销
- SQL 没有定义物化视图标准 [p.116]

11.5 视图的优缺点 [p.117]

优点：数据保密、简化数据查询操作、保证数据逻辑独立性、着重于特定数据、自定义数据、导出和导入数据、跨服务器组合分区数据。

缺点：更新视图数据时实际是对基表更新，某些视图不能更新数据。

11.6 创建视图的限制 [p.118–119]

- 只能在当前数据库中创建视图 [p.118]
- 可以在其他视图的基础上创建视图 [p.118]
- 不能将规则或 DEFAULT 定义与视图相关联 [p.118]
- 不能将 AFTER 触发器与视图相关联，只有 INSTEAD OF 触发器可以 [p.118]
- 定义视图的查询不能包含 COMPUTE 子句、COMPUTE BY 子句或 INTO 关键字 [p.119]
- 不能包含 ORDER BY 子句（除非有 SELECT TOP 子句） [p.119]
- 不能为视图定义全文索引 [p.119]
- 不能创建临时视图 [p.119]

11.7 通过视图更新数据 [p.127–129]

可更新视图的条件 [p.129]：

- 任何修改 (UPDATE/INSERT/DELETE) 都只能引用一个基表的列 (FROM 只有一个关系)
- SELECT 中只含关系的属性名，不含表达式、聚集、DISTINCT 声明
- 查询中不含 GROUP BY、HAVING 子句
- 视图中被修改的列必须直接引用表列中的基础数据

大多数 SQL 实现只允许对简单视图（无聚合，定义在单个关系上）进行更新 [p.128]

示例——通过视图插入需要映射到基表 [p.127]：

```
CREATE VIEW branch_loan AS
SELECT loan_number, branch_name
FROM loan;

INSERT INTO branch_loan VALUES ('L-37', '中行铁道支行');
-- 实际转换为：INSERT INTO loan VALUES ('L-37', '中行铁道支行', NULL);
```

有些视图更新是二义性的：如 all_customer 视图包含 UNION，插入时需要选择是 loan 还是 account [p.128]

12 用户自定义类型与域 [p.130–131]

12.1 CREATE TYPE / CREATE DOMAIN [p.130]

- CREATE TYPE: SQL 标准创建用户自定义类型 [p.130]

```
CREATE TYPE Dollars AS NUMERIC(12,2) FINAL;
```

- CREATE DOMAIN: SQL-92 创建用户自定义域类型 [p.130]

```
CREATE DOMAIN person_name CHAR(20) NOT NULL;
```

- Types 和 Domain 类似，但 Domain 可以有约束（如 NOT NULL） [p.130]
- SQL Server 使用存储过程: sp_addtype, sp_droptype [p.130]

12.2 域的约束 [p.131]

- 不同域之间不能直接赋值或比较（如 Dollars 和 Pounds），需要用 CAST 转换 [p.131]
- CAST(r.A AS Pounds) 进行类型转换 [p.131]

12.3 大对象类型 [p.132]

- BLOB (Binary Large Object): 二进制大对象，用于照片、视频、CAD 文件等 [p.132]
- CLOB (Character Large Object): 字符大对象 [p.132]
- 查询返回大对象时，返回的是指针而非对象本身 [p.132]

13 完整性约束 (Integrity Constraints) [p.133–144]

13.1 概述 [p.133]

完整性约束通过确保对数据库的授权更改不会导致数据一致性丢失，从而防止对数据库的意外损坏 [p.133]。

13.2 NOT NULL 约束 [p.134–135]

- 确保属性值不能为空
- 例: `branch_name CHAR(15) NOT NULL` [p.135]
- 域级 NOT NULL: `CREATE DOMAIN Dollars NUMERIC(12,2) NOT NULL` [p.135]

13.3 PRIMARY KEY 约束 [p.134]

- 主键自动确保 NOT NULL (SQL-92 起) [p.11]
- 可指定约束名: `CONSTRAINT pk_sc PRIMARY KEY (stud_ID, course_ID)` [p.134]
- 增加约束: `ALTER TABLE branch ADD CONSTRAINT pk_chka CHECK(asset > 1200);` [p.134]
- 删除约束: `ALTER TABLE branch DROP CONSTRAINT pk_chka;` [p.134]

13.4 FOREIGN KEY 约束 [p.134, 139–141]

- 确保引用完整性 (Referential Integrity) [p.139]
- 默认情况下，外键引用被参照表的主键属性 [p.139]
- 完整示例 [p.141]:

```
CREATE TABLE account (  
    account_number CHAR(10),  
    branch_name    CHAR(15),  
    balance        INTEGER,  
    PRIMARY KEY (account_number),  
    FOREIGN KEY (branch_name) REFERENCES branch  
);
```

注意：外键属性名与被参照表的主键属性名可以不同，但数据类型必须相同 [p.12]

13.5 UNIQUE 约束 [p.136]

- `UNIQUE (A1, A2, ..., Am)`: 指定候选键 (Candidate Key) [p.136]
- 候选键允许为 null (与主键不同) [p.136]

13.6 CHECK 约束 [p.137–138]

`CHECK (P)`, 其中 P 是谓词 [p.134]:

```
-- 表级 CHECK  
CREATE TABLE branch (  
    branch_name CHAR(15),
```

```
branch_city CHAR(30),
assets      INT,
PRIMARY KEY (branch_name),
CHECK (assets >= 0)
);

-- 域级 CHECK
CREATE DOMAIN hourly_wage NUMERIC(5,2)
CONSTRAINT value_test CHECK(value >= 4.00);
```

13.7 断言 (Assertion) [p.142–144]

断言是表达我们希望数据库始终满足的条件的谓词 [p.142]:

```
CREATE ASSERTION <assertion-name> CHECK <predicate>;
```

- 每次更新都要检测是否违反断言，增加负载，应谨慎使用 [p.142]
- ”对所有 X, P(X)”的实现方式：使用 NOT EXISTS X SUCH THAT NOT P(X) [p.142]

断言示例——每笔贷款至少有一位借款人持有最低余额 \$1000 的账户 [p.143]:

```
CREATE ASSERTION balance_constraint CHECK
(
  NOT EXISTS (
    SELECT * FROM loan
    WHERE NOT EXISTS (
      SELECT * FROM borrower, depositor, account
      WHERE loan.loan_number = borrower.loan_number
        AND borrower.customer_name = depositor.customer_name
        AND depositor.account_number = account.account_number
        AND account.balance >= 1000
    )
  )
);
```

14 授权 (Authorization) [p.145–152]

14.1 授权类型 [p.145]

对数据库部分的授权形式 [p.145]:

- **Read:** 允许读取, 不能修改
- **Insert:** 允许插入新数据
- **Update:** 允许修改, 不能删除
- **Delete:** 允许删除

对数据库模式的授权形式 [p.145]:

- **Index:** 允许创建和删除索引
- **Resources:** 允许创建新关系
- **Alteration:** 允许添加或删除属性
- **Drop:** 允许删除关系

14.2 GRANT —授予权限 [p.146–147, 149–150]

```
GRANT <privilege list>  
ON <relation name or view name>  
TO <user list>;
```

用户列表可以是: user-id、PUBLIC (所有有效用户)、或角色 [p.146]

权限类型 [p.147]:

- **SELECT:** 允许读取/查询
- **INSERT:** 允许插入元组
- **UPDATE:** 允许更新
- **DELETE:** 允许删除
- **ALL PRIVILEGES:** 所有权限的简写形式

带有传播权限的授权 [p.150]:

```
GRANT INSERT ON TABLE SC TO U5 WITH GRANT OPTION;  
-- U5 可以将此权限授予 U6:  
GRANT INSERT ON TABLE SC TO U6 WITH GRANT OPTION;  
-- U6 可以授予 U7 (但 U7 不能再传播):  
GRANT INSERT ON TABLE SC TO U7;
```

注意: 授权给视图并不意味着授权给基表; 有权才可授权 [p.146]

14.3 REVOKE —收回权限 [p.148, 151–152]

```
REVOKE <privilege list>  
ON <relation name or view name>  
FROM <user list>;
```

级联收回 [p.152]: 收回权限的操作会级联。如果 U5 将 INSERT 权限授予 U6, U6 再授予 U7, 收回 U5 的 INSERT 权限时会自动收回 U6 和 U7 的 INSERT 权限。但如果 U6 或 U7 还从其他用户处获得 INSERT 权限, 则仍具有此权限。系统只收回直接或间接从 U5 处获得的权限 [p.152]。

15 SQL 流程控制 [p.153–175]

15.1 变量 [p.153–160]

15.1.1 局部变量 [p.154–157]

- 作用范围限制在程序内部，程序结束后消失 [p.154]
- 引用时名称前加 @ 符号，需先用 DECLARE 定义 [p.154]
- 声明语法: DECLARE @local_variable data_type [, ...n] [p.155]
- 赋值: SET @local_variable = expression 或 SELECT @local_variable = expression [p.155]

```
DECLARE @myvar CHAR(20);
SELECT @myvar = 'This is a test';
SELECT @myvar;

-- 通过查询赋值
DECLARE @rows INT;
SET @rows = (SELECT COUNT(*) FROM s);
```

15.1.2 全局变量 [p.158–160]

- 系统内部使用的变量，任何程序均可调用 [p.158]
- 引用时加 @@ 前缀 [p.158]
- 两类：连接相关（如 @@rowcount）和系统信息相关（如 @@version） [p.158]
- 局部变量名不能与全局变量名相同 [p.159]

15.2 IF...ELSE 语句 [p.161–162]

```
IF Boolean_expression
{ sql_statement | statement_block }
[ ELSE
{ sql_statement | statement_block } ]
```

15.3 BEGIN...END 语句块 [p.163]

将多个 SQL 语句组合成一个语句块 [p.163]:

```
BEGIN
    sql_statement | statement_block
END
```

15.4 CASE 语句 [p.167–168]

两种形式 [p.167]:

- 简单 CASE: CASE input_expression WHEN ... THEN ... END
- 搜索 CASE: CASE WHEN Boolean_expression THEN ... END

15.5 WHILE 循环 [p.170–171]

```
WHILE Boolean_expression  
{ sql_statement | statement_block }  
[ BREAK ]  
[ CONTINUE ]
```

15.6 GOTO / RETURN [p.172–175]

- GOTO label: 跳到指定标识符处继续执行，标识符以”:" 结尾 [p.172]
- RETURN [integer_expression]: 无条件终止查询/存储过程/批处理 [p.174]

16 存储过程 (Stored Procedures) [p.176–213]

16.1 概念与特点 [p.177–181]

- 存储过程是存储在数据库中的可执行特定工作的一组 SQL 代码程序段 [p.177]
- 与函数的区别 [p.177–179]:
 - 函数有且只有一个返回值，可直接在表达式中嵌入调用
 - 存储过程可以没有返回值，也可以有任意个输出参数，必须单独调用
 - 函数限制较多（不能用临时表，只能用表变量）
 - 存储过程实现的功能更复杂，函数功能针对性更强

优点 [p.181]:

- 提高系统性能：执行一次后代码驻留在高速缓存中，再次执行直接调用已编译代码
- 减少应用服务器与数据库的数据传输
- 实现模块化设计：创建后可多次调用，业务规则改变只需修改存储过程
- 确保数据库安全：用户不需要直接访问数据库对象

缺点 [p.183]:

- 可移植性差（与特定 DBMS 绑定）
- 调试困难、管理和维护困难
- 不适合高并发场景、影响系统灵活性
- 代码可读性差

16.2 创建存储过程 [p.185–189]

```
CREATE PROCEDURE sp_name ([proc_parameter[,...]])  
[characteristic ...] routine_body
```

参数类型 [p.186]:

- IN: 把数据从外部传递给存储过程
- OUT: 从存储过程内部返回值给外部
- INOUT: 兼具以上两种功能

示例 [p.188]:

```
DELIMITER //  
CREATE PROCEDURE sp_test (IN inparms INT, OUT outparams VARCHAR(32))  
BEGIN  
    DECLARE var CHAR(10);  
    DECLARE num INT;  
    IF inparms = 1 THEN  
        SET var = 'hello';  
    ELSE  
        SET var = 'world';  
    END IF;  
    INSERT INTO t1(name) VALUES (var);  
    SELECT COUNT(*) FROM t1 INTO num;  
    SELECT name FROM t1 LIMIT 1 INTO outparams;
```

```
END //
DELIMITER ;
```

16.3 变量声明与赋值 [p.189–191]

- DECLARE var_name [, ...] type [DEFAULT value] [p.189]
- 提供默认值需包含 DEFAULT 子句, 否则初始值为 NULL
- 局部变量作用范围在 BEGIN...END 块内 [p.189]
- SET 赋值: SET var_name = expr [p.190]
- SELECT ... INTO 赋值: 将选择的列值存储到变量, 只能取回单行 [p.191]

16.4 流程控制 (MySQL) [p.195–209]

- 顺序、分支、循环 [p.195]
- IF 语句、CASE 语句 [p.196–203]
- WHILE 循环 [p.204–207]
- LOOP ... END LOOP [p.208]
- ITERATE (迭代) 语句 [p.209]

16.5 DELIMITER [p.192–194]

MySQL 中修改语句结束标志 [p.192]:

- 默认以分号 (;) 为结束标志
- 存储过程体中可能包含多个以分号结尾的 SQL 语句
- 使用 DELIMITER //或 DELIMITER \$\$ 改变结束符
- 用 DELIMITER ; 恢复默认 [p.194]

16.6 注释与权限 [p.210]

- 单行注释: --
- 多行注释: /* ... */
- 权限: CREATE ROUTINE (建立)、ALTER ROUTINE (编辑删除)、EXECUTE (执行) [p.210]

16.7 条件处理 (HANDLER) [p.211–213]

```
DECLARE handler_type HANDLER FOR
    condition_value[,...] sp_statement;
```

- CONTINUE: 处理程序语句之后继续执行
- EXIT: 终止当前 BEGIN...END 复合语句的执行
- 条件类型: SQLWARNING (01 开头)、NOT FOUND (02 开头)、SQLEXCEPTION (其他) [p.212]

17 触发器 (Triggers) [p.214–229]

17.1 触发器概念 [p.214]

- 触发器是一种特殊的存储过程 [p.214]
- 一组 SQL 语句的集合，作为表的一部分被创建 [p.214]
- 当预定义事件（插入/删除/更新）发生时自动执行，无需用户调用 [p.214]
- 常用于保护表中数据或实现数据完整性 [p.214]

17.2 触发器语法分析 [p.215–223]

17.2.1 创建限制 [p.215]

- 只能在永久表上创建触发器，不能对临时表或视图创建 [p.215]

17.2.2 触发器名称 [p.216]

- 在当前数据库中必须具有唯一的名称
- 在不同数据库中可以同名

17.2.3 触发时间 [p.217]

- AFTER: 在激活触发器的语句执行之后触发
- BEFORE: 在语句执行之前触发，可用于验证新数据是否满足使用限制

17.2.4 触发事件 [p.218]

- INSERT: 插入新行时激活
- DELETE: 删除行时激活
- UPDATE: 更改行时激活
- 对同一个表相同触发时间的相同触发事件，只能定义一个触发器（MySQL 限制） [p.219]

17.2.5 FOR EACH ROW [p.220]

对于受触发器事件影响的每一行都要激活触发器的动作。

17.2.6 OLD 和 NEW 引用 [p.221–223]

- OLD: 关联被删除或被更新前的记录
- NEW: 关联被插入的记录或被更新后的记录
- INSERT 语句: 只有 NEW 合法
- DELETE 语句: 只有 OLD 合法
- UPDATE 语句: 可以与 NEW 或 OLD 同时使用
- 建议不使用 SELECT 语句（阻止从触发器返回结果） [p.223]

17.3 触发器示例 [p.224–228]

当删除学生信息表 student 中的记录时，成绩表 sc 中对应的成绩也应删除 [p.224]:

```
DROP TRIGGER IF EXISTS tr_stu_delete;
DELIMITER //
CREATE TRIGGER tr_stu_delete BEFORE DELETE ON student
FOR EACH ROW
BEGIN
    DELETE FROM sc WHERE s_id = OLD.s_id;
END//
DELIMITER ;
```

注意——BEFORE vs AFTER [p.227]:

- 使用 BEFORE 是因为 Student 为主表，sc 为子表，要删除主表数据必须先删除子表数据
- 使用 AFTER 创建触发器虽然成功，但执行 DELETE 时会报错（违反了外键约束），触发器不起作用

17.4 触发器的缺点 [p.229]

- 隐藏逻辑：增加了代码维护难度和理解成本
- 性能影响：增加数据库负载，大量数据时性能下降
- 难以调试：自动触发，没有参数，不需要显示调用
- 多个触发器复杂性：执行顺序复杂，嵌套容易导致死锁
- 建议：尽量考虑使用存储过程替代触发器 [p.229]

18 游标 (Cursor) [p.230–235]

18.1 游标概念 [p.230–231]

- 游标是一种处理数据的方法，提供在结果集中向前或向后浏览数据的能力 [p.230]
- 基于逐行操作结果集的方法，对 SELECT 查询结果集中的记录行逐行处理 [p.230]
- 作为面向集合的 DBMS 和面向行的程序设计之间的桥梁 [p.231]

18.2 游标使用步骤 [p.232–234]

五步走 [p.232]:

1. 声明/创建游标
2. 打开游标
3. 推进游标指针从结果集中提取数据（逐行处理）
4. 关闭游标
5. 释放游标

语法 [p.233–234]:

```
-- 声明游标
DECLARE cursor_name CURSOR FOR select_statement
    [FOR {READ ONLY | UPDATE [OF column_name_list]}];

-- 打开游标
OPEN cursor_name;

-- 读取数据
FETCH [[NEXT|PRIOR|FIRST|LAST]FROM] cursor_name
    [INTO fetch_target_list];

-- 通过游标删除
DELETE [FROM] {table_name | view_name}
WHERE CURRENT OF cursor_name;

-- 通过游标更新
UPDATE {table_name | view_name}
SET column_name1 = expression1 [, ...]
WHERE CURRENT OF cursor_name;

-- 关闭游标（不用再声明可再打开）
CLOSE cursor_name;

-- 释放游标（释放后不能再打开，需重新声明）
DEALLOCATE CURSOR cursor_name;
```

18.3 游标示例 [p.235]

```
DELIMITER //
```

```
CREATE PROCEDURE read_data()
BEGIN
    DECLARE done INT DEFAULT FALSE;
    DECLARE id INT;
    DECLARE name VARCHAR(255);
    DECLARE cur CURSOR FOR SELECT s_id, s_name FROM student;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

    OPEN cur;
    read_loop: LOOP
        FETCH cur INTO id, name;
        IF done THEN
            LEAVE read_loop;
        END IF;
        -- 处理游标读取到的数据
        SELECT CONCAT(id, ' : ', name);
    END LOOP;
    CLOSE cur;
END//
DELIMITER ;
```


19 综合示例与补充 [p.73–91, 55–56]

19.1 学生数据库完整示例 [p.73–76]

学生基本信息表 stud_info [p.74–75]:

```
CREATE TABLE stud_info (  
  stud_id CHAR(10) NOT NULL,  
  name NVARCHAR(4) NOT NULL,  
  birthday DATETIME,  
  gender NCHAR(1),  
  address NVARCHAR(20),  
  telcode CHAR(12),  
  zipcode CHAR(6),  
  mark DECIMAL(3,0)  
);
```

教师信息表与成绩表 [p.76]:

```
CREATE TABLE teacher_info (  
  teacher_id CHAR(6) NOT NULL,  
  name NVARCHAR(4) NOT NULL,  
  gender NCHAR(1),  
  age INT,  
  tech_title NVARCHAR(5),  
  telephone VARCHAR(12),  
  salary DECIMAL(7,2),  
  course_id CHAR(10)  
);  
  
CREATE TABLE stud_grade (  
  stud_id CHAR(10) NOT NULL,  
  name NVARCHAR(4) NOT NULL,  
  course_id CHAR(10),  
  grade DECIMAL(4,1)  
);
```

19.2 综合 SQL 练习示例 [p.55–56]

```
-- a. 找出Comp. Sci.系3学分课程的标题  
SELECT title  
FROM course  
WHERE dept_name = 'Comp. Sci.' AND credits = 3;  
  
-- b. 找出所有工资最高的老师（可能多个）  
SELECT ID, name  
FROM instructor  
WHERE salary IN (SELECT MAX(salary) FROM instructor);
```

```
-- c. 统计2009年秋季每个section的注册人数
SELECT course_id, sec_id, COUNT(ID)
FROM section NATURAL JOIN takes
WHERE semester = 'Autumn' AND year = 2009
GROUP BY course_id, sec_id;
```

19.3 GROUP BY + HAVING 综合 [p.88-89]

```
-- 统计软件工程专业学生的平均入学成绩
-- （学号中3-4位为院系，5-6位为专业=01）
SELECT ...
FROM stud_info
WHERE SUBSTRING(stud_id, 5, 2) = '01'
GROUP BY ...

-- 按职称分组统计"讲师"的平均年龄
SELECT tech_title, AVG(age)
FROM teacher_info
GROUP BY tech_title
HAVING tech_title = N'讲师';
```

20 易错点与考点总结

20.1 DDL 相关易错点

1. **主键 vs 候选键**: 主键自动 NOT NULL (SQL-92 起), 候选键 (UNIQUE) 允许 NULL
2. **外键域名**: 外键与被参照主键的列名可以不同, 但域 (数据类型) 必须相同
3. **ALTER TABLE 限制**: 很多数据库不支持删除属性列
4. **CHECK 约束**: 可以用 CONSTRAINT 命名约束, 便于后续修改或删除

20.2 DML 查询易错点

1. **DISTINCT 位置**: 紧跟在 SELECT 之后, 作用于所有选择的列
2. **WHERE vs HAVING (高频考点)**:
 - WHERE 在分组前过滤行 (不能用聚合函数)
 - HAVING 在分组后过滤组 (可以使用聚合函数)
3. **GROUP BY 规则**: SELECT 中出现的非聚合属性必须在 GROUP BY 中出现
4. **NULL 判断**: 必须用 IS NULL/IS NOT NULL, 不能用 = NULL 或 <> NULL
5. **COUNT(*) vs COUNT(列名)**: COUNT(*) 统计所有行, COUNT(列名) 忽略 NULL
6. **SELECT 语句子句顺序 (固定不可变)**:

SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY

20.3 子查询易错点

1. **相关子查询 vs 不相关子查询**: 相关子查询效率较低, 对外层每行都要执行
2. **= SOME 等价于 IN**, 但 **<> SOME 不等于 NOT IN**
3. **<> ALL 等价于 NOT IN**, 但 **= ALL 等价于 IN**
4. **实现”全部”语义**: 使用 NOT EXISTS + EXCEPT 的双重否定模式 ($X \subseteq Y \Leftrightarrow X - Y = \emptyset$)
5. **NOT IN 与 NULL**: 如果子查询结果包含 NULL, NOT IN 可能返回空结果

20.4 连接易错点

1. **自然连接陷阱**: 同名的无关属性会被错误地等同起来, 应使用 USING 或 ON 明确指定
2. **外连接空值检测**: 全外连接后用 IS NULL 检测不匹配的行
3. **自连接需要别名**: 同一表的不同副本必须有不同的别名

20.5 数据修改易错点

1. **DELETE 中的子查询**: SQL 先计算子查询找出所有要删除的元组, 再一次性删除
2. **INSERT ... SELECT**: SELECT 先完全求值, 再插入结果
3. **UPDATE 顺序**: 多条 UPDATE 语句执行顺序可能影响结果, 建议用 CASE 合并
4. **视图更新限制**: 大多数 SQL 实现只允许对简单视图 (单表、无聚合、无 DISTINCT、无 GROUP BY) 更新

20.6 其他考点

1. **GRANT WITH GRANT OPTION:** 允许被授权者继续授权; REVOKE 会级联收回
2. **三值逻辑:** SQL 中的布尔值为 TRUE/FALSE/UNKNOWN, 涉及 NULL 的比较返回 UNKNOWN
3. **聚合函数忽略 NULL:** 除 COUNT(*) 外, 所有聚合函数忽略 NULL
4. **BLOB vs CLOB:** BLOB 存二进制数据, CLOB 存字符数据
5. **视图展开:** 非递归视图可通过替换展开到底层基表
6. **游标步骤:** DECLARE → OPEN → FETCH → CLOSE → DEALLOCATE
7. **BEFORE vs AFTER 触发器:** 删除主表数据时, BEFORE 触发器可在违反外键约束前先删除子表数据

21 SQL 关键词速查表

类别	关键词	功能
DDL	CREATE TABLE/DATABASE/VIEW/INDEX	创建表/数据库/视图/索引
DDL	ALTER TABLE	修改表结构（添加/删除/修改列）
DDL	DROP TABLE/DATABASE/VIEW	删除表/数据库/视图
DML	SELECT ... FROM ... WHERE	查询数据
DML	INSERT INTO ... VALUES	插入元组
DML	UPDATE ... SET ... WHERE	更新元组
DML	DELETE FROM ... WHERE	删除元组
DCL	GRANT ... ON ... TO	授予权限
DCL	REVOKE ... ON ... FROM	收回权限
约束	PRIMARY KEY	主键约束
约束	FOREIGN KEY ... REFERENCES	外键约束（引用完整性）
约束	NOT NULL	非空约束
约束	UNIQUE	候选键约束（允许 NULL）
约束	CHECK	域/表级检查约束
查询修饰	DISTINCT	消除重复行
查询修饰	AS (别名)	重命名属性或关系
查询修饰	ORDER BY ... ASC/DESC	排序
连接	NATURAL JOIN	自然连接（自动匹配同名列）
连接	INNER JOIN ... ON	内连接
连接	LEFT/RIGHT/FULL OUTER JOIN	左/右/全外连接
集合	UNION [ALL]	并集
集合	INTERSECT [ALL]	交集
集合	EXCEPT [ALL]	差集
聚合	COUNT/SUM/AVG/MAX/MIN	聚合函数
聚合	GROUP BY	分组
聚合	HAVING	分组过滤
子查询	IN / NOT IN	集合成员测试
子查询	EXISTS / NOT EXISTS	存在性测试
子查询	SOME / ALL	集合比较
子查询	UNIQUE	重复性测试
模式匹配	LIKE / NOT LIKE	字符串模式匹配（% _）
空值	IS NULL / IS NOT NULL	空值判断
事务	COMMIT / ROLLBACK	提交/回滚